

CDAC FUTURE SKILLS PROGRAMME

Demystifying Android Application Security

Day 01 - Assignment

Name : Vimal Mudalagi

Institution : Digital University Kerala

Programme : M.Sc. Computer Science (Specialization: Cyber Security)

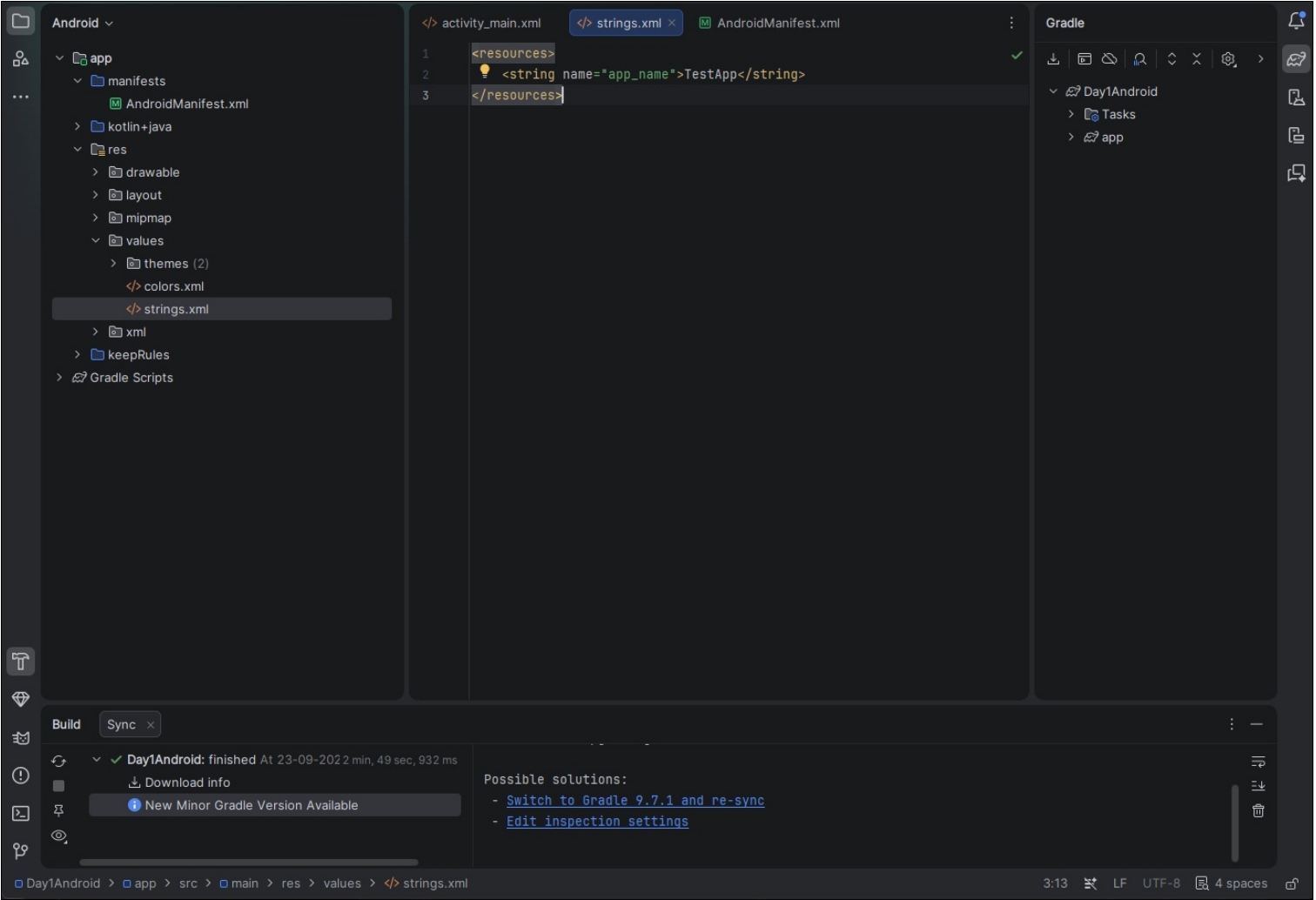
Task 1 - Change the Application Name to "TestApp"

The application name was changed by editing the **app_name** string resource in **strings.xml**, which is referenced by the **android:label** attribute in **AndroidManifest.xml**.

res/values/strings.xml
<pre><resources> <string name="app_name">TestApp</string> </resources></pre>
AndroidManifest.xml
<pre><application android:allowBackup="true" android:dataExtractionRules="@xml/data_extraction_rules" android:fullBackupContent="@xml/backup_rules" android:icon="@mipmap/ic_launcher" android:label="@string/app_name" android:roundIcon="@mipmap/ic_launcher_round" android:supportRtl="true" android:theme="@style/Theme.Day1Android"> <activity android:name=".MainActivity" android:exported="true" android:windowSoftInputMode="adjustResize"> <intent-filter> <action android:name="android.intent.action.MAIN" /> <category android:name="android.intent.category.LAUNCHER" /> </intent-filter> </activity> </application></pre>

Screenshot 1

strings.xml showing the app_name resource set to "TestApp".



Screenshot 2

AndroidManifest.xml showing android:label referencing @string/app_name.

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools">
4
5     <application
6         android:allowBackup="true"
7         android:dataExtractionRules="@xml/data_extraction_rules"
8         android:fullBackupContent="@xml/backup_rules"
9         android:icon="@mipmap/ic_launcher"
10        android:label="@string/app_name"
11        android:roundIcon="@mipmap/ic_launcher_round"
12        android:supportsRtl="true"
13        android:theme="@style/Theme.Day1Android">
14        <activity
15            android:name=".MainActivity"
16            android:exported="true"
17            android:windowSoftInputMode="adjustResize">
18            <intent-filter>
19                <action android:name="android.intent.action.MAIN" />
20
21                <category android:name="android.intent.category.LAUNCHER" />
22            </intent-filter>
23        </activity>
24    </application>
25
26 </manifest>
```

Task 2 - Add a Logo / Icon to the Application

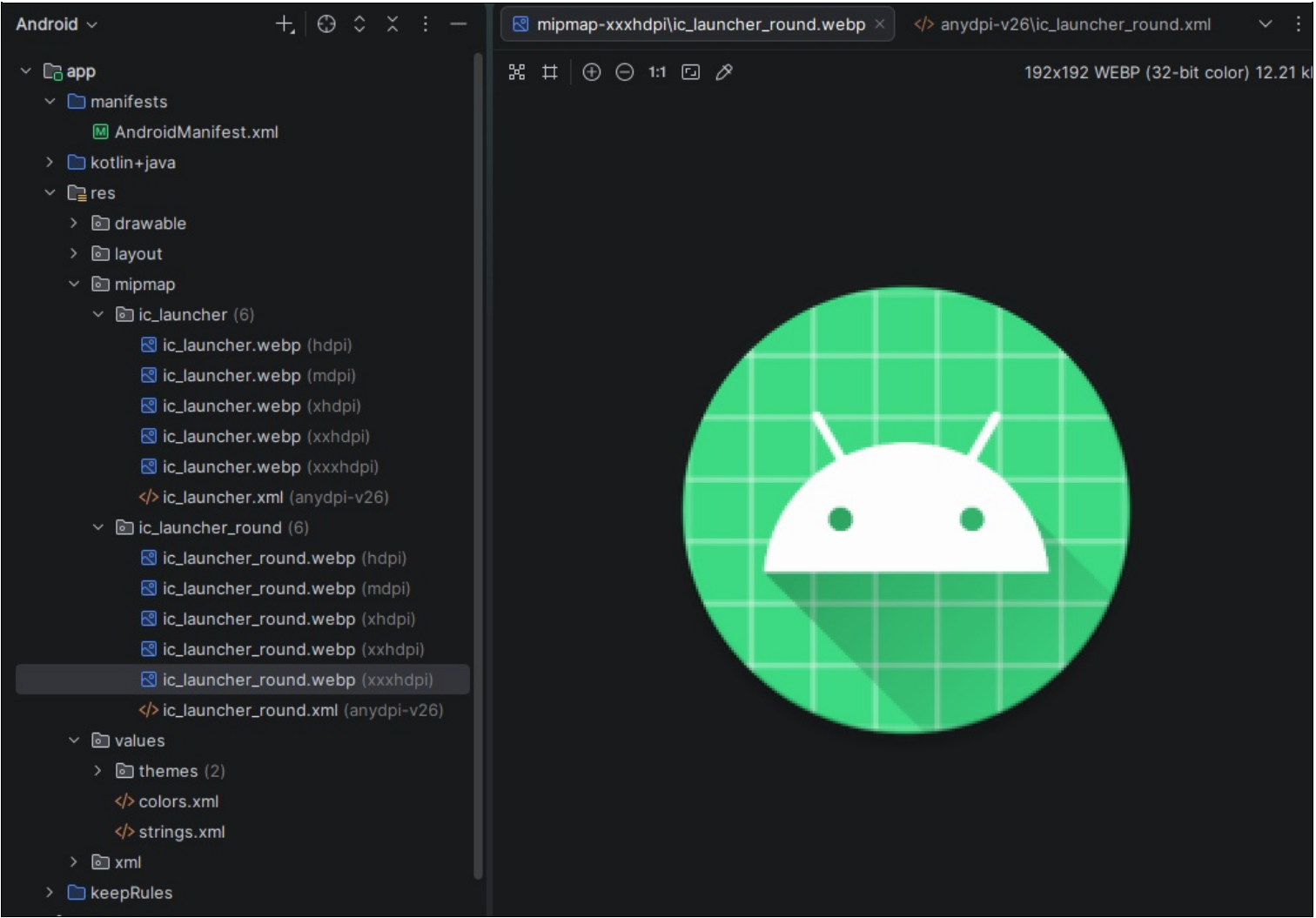
A new application icon was generated using Android Studio's **New → Image Asset** wizard. The wizard produced the launcher icon in both square (**ic_launcher**) and round (**ic_launcher_round**) formats across all mipmap density buckets (hdpi, mdpi, xhdpi, xxhdpi, xxxhdpi).

```
res/mipmap-anydpi-v26/ic_launcher_round.xml

<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
  <background android:drawable="@drawable/ic_launcher_background" />
  <foreground android:drawable="@drawable/ic_launcher_foreground" />
</adaptive-icon>
```

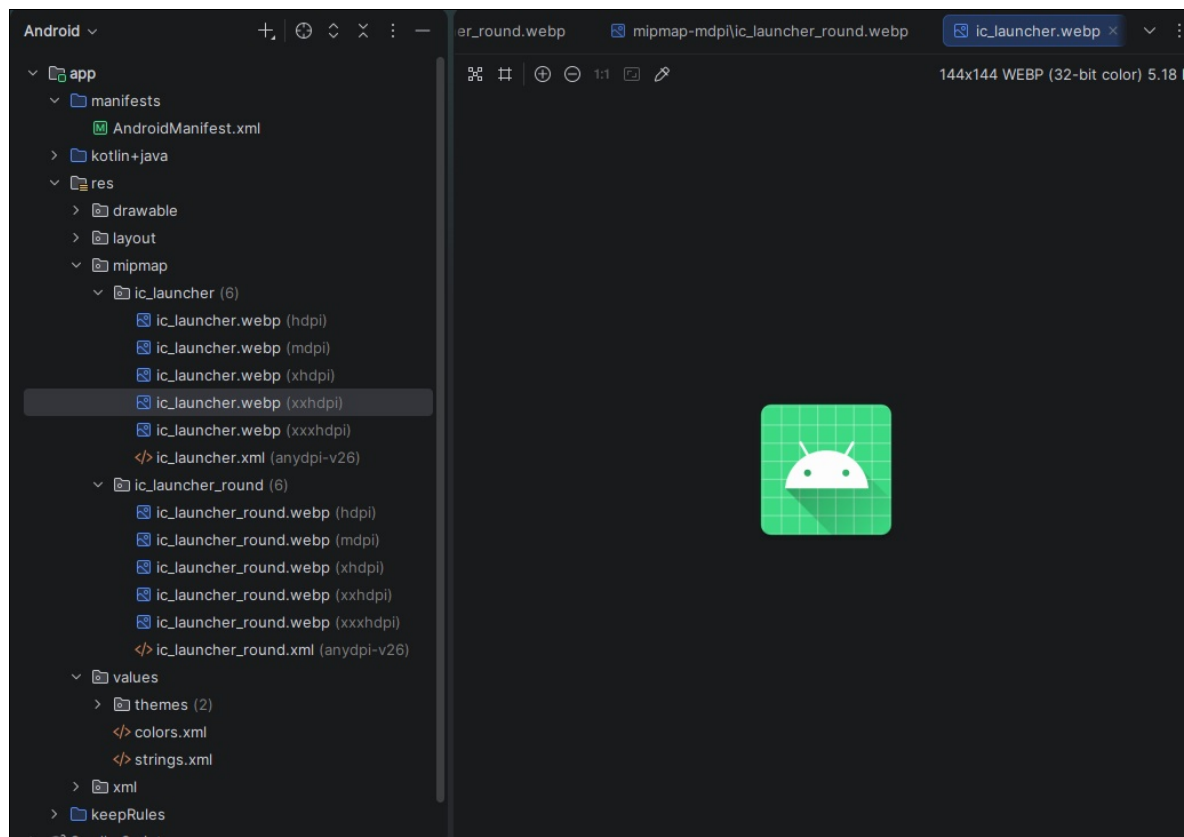
Screenshot 1

Image Asset Studio preview of the generated ic_launcher_round icon (192x192, xxxhdpi).



Screenshot 2

The generated icon at xhdpi density (144x144) shown in the resource preview pane.



Task 3 - Display "Welcome to Android Development" at Screen Center

A **LinearLayout** with **android:gravity="center"** was used to position a single **TextView** in the exact center of the screen, both horizontally and vertically.

```
res/layout/activity_main.xml

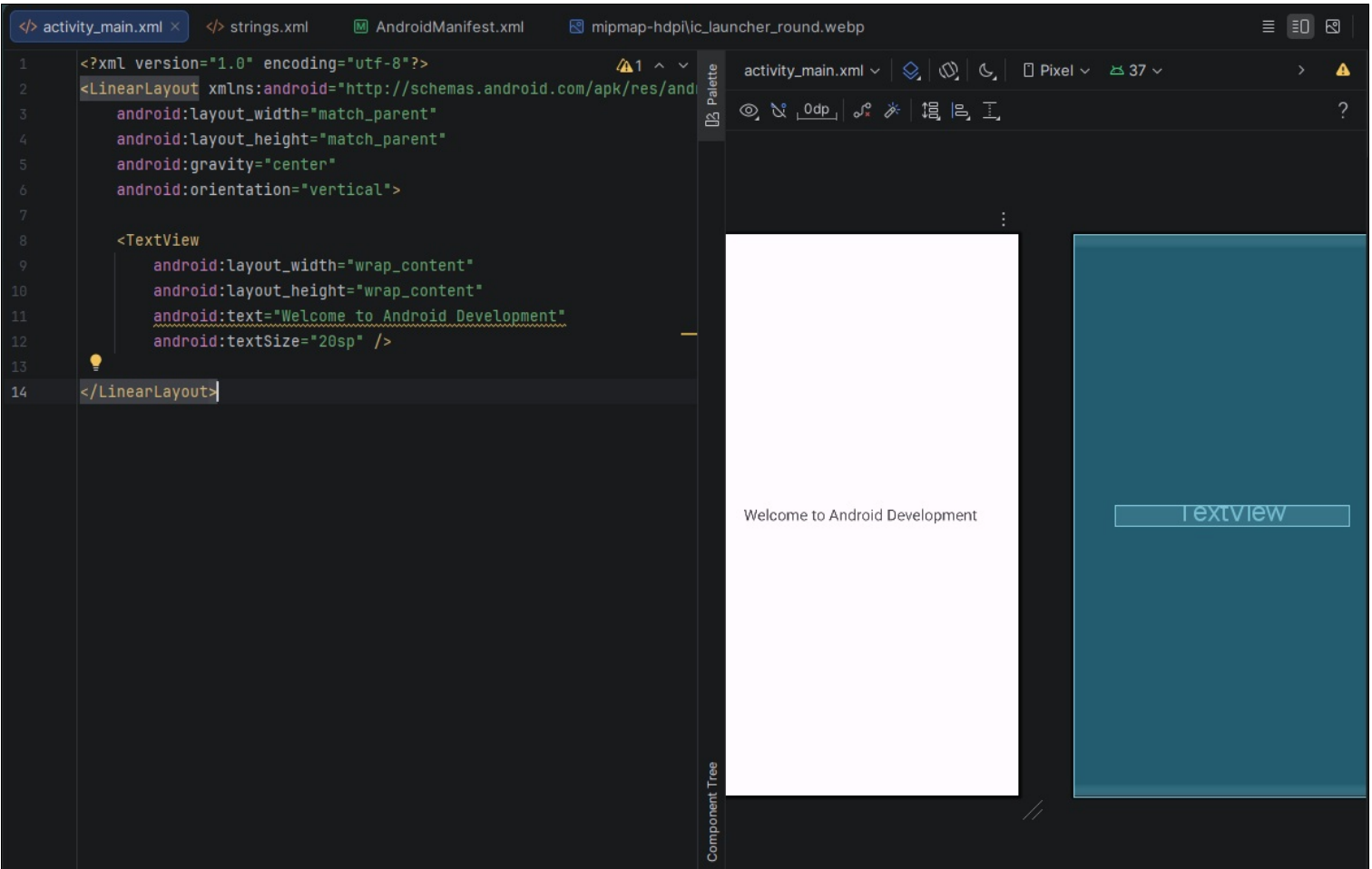
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome to Android Development"
        android:textSize="20sp" />

</LinearLayout>
```

Screenshot

Layout preview and design view showing the **TextView** centered on the screen.



Task 4 - Set the Screen Background Colour to Light Blue

The **android:background** attribute was added to the root LinearLayout with a hex colour value to change the screen background to light blue.

```
res/layout/activity_main.xml

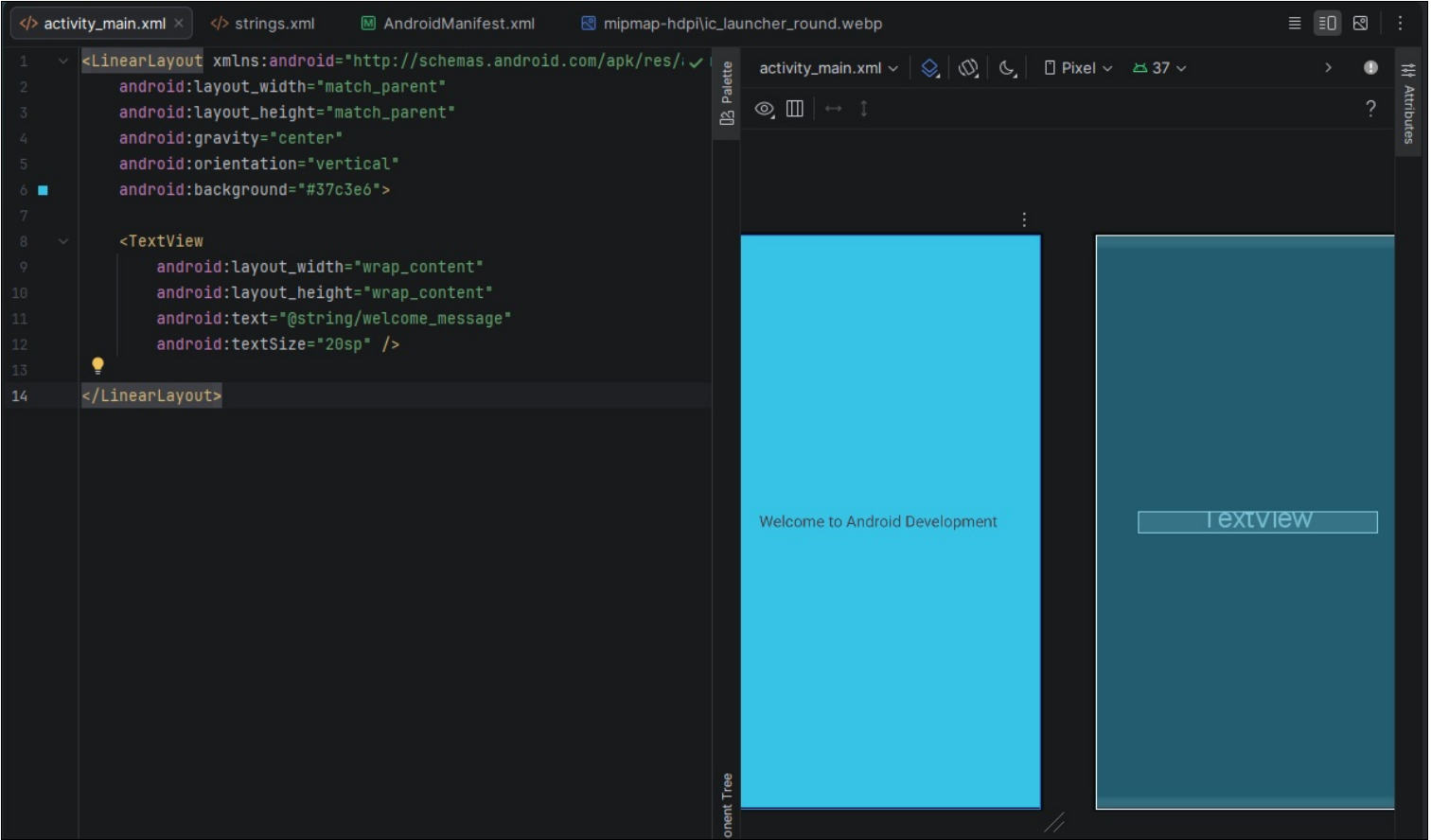
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:background="#37c3e6">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/welcome_message"
        android:textSize="20sp" />

</LinearLayout>
```

Screenshot

Layout preview showing the light blue (#37c3e6) background applied to the screen.



Task 5 - Greeting Application

An EditText captures the user's name, a Button triggers the action via **setOnClickListener()**, and the entered name is read and displayed inside a TextView as a greeting message.

```
res/layout/activity_main.xml<EditText
    android:id="@+id/nameInput"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter your name" />

<Button
    android:id="@+id/greetButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Greet" />

<TextView
    android:id="@+id/greetingText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:textSize="20sp" />
```

```
MainActivity.javapackage com.example.day1android;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import android.widget.EditText;
import android.widget.Button;
import android.widget.TextView;

public class MainActivity extends AppCompatActivity {

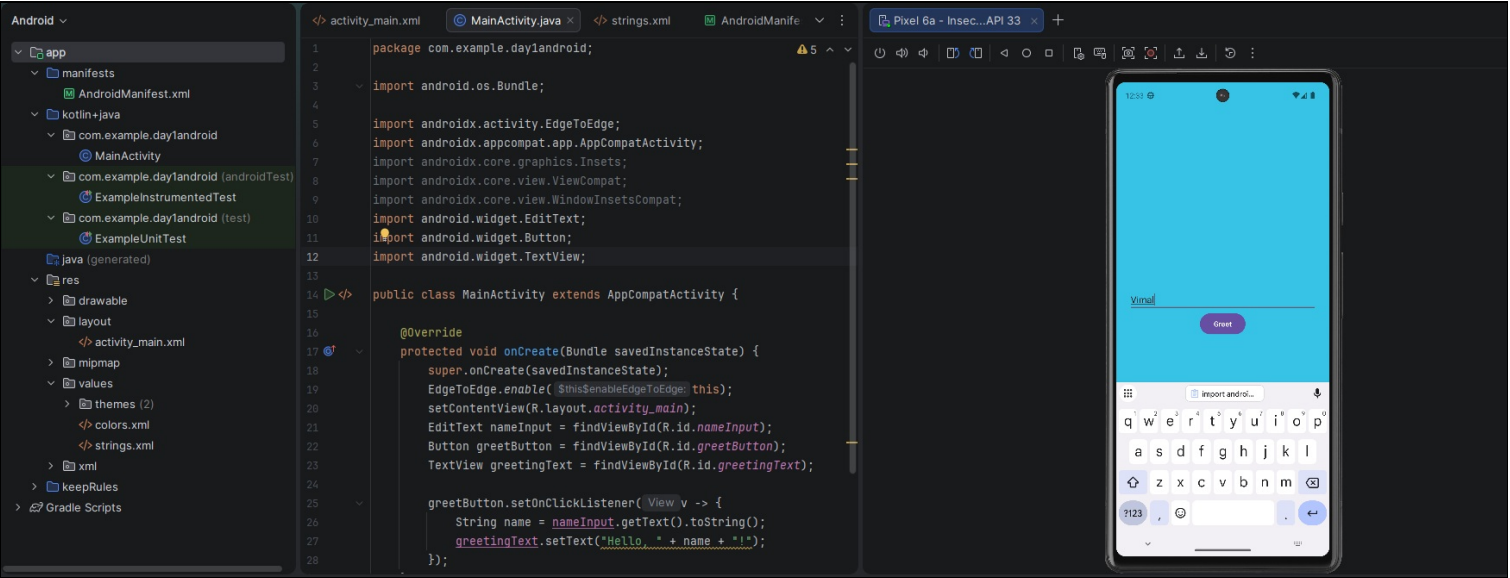
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        EditText nameInput = findViewById(R.id.nameInput);
        Button greetButton = findViewById(R.id.greetButton);
        TextView greetingText = findViewById(R.id.greetingText);

        greetButton.setOnClickListener(view -> {
            String name = nameInput.getText().toString();
            greetingText.setText("Hello, " + name + "!");
        });
    }
}
```

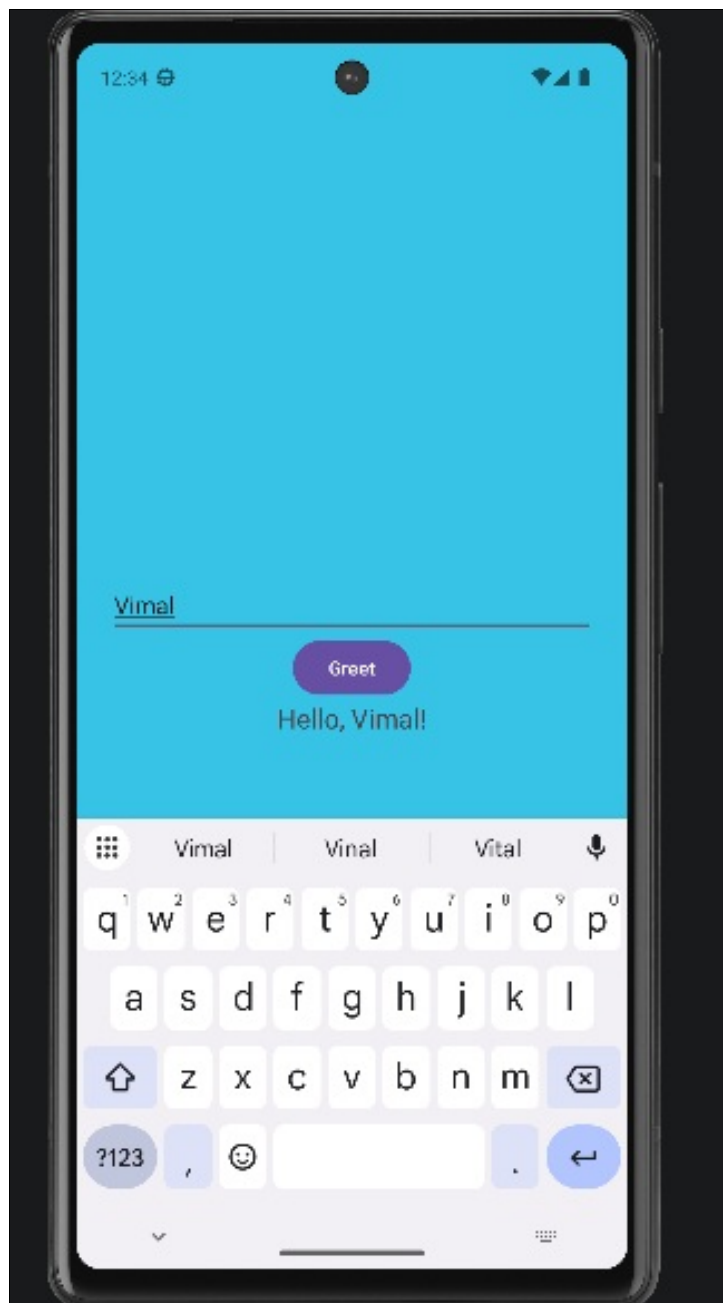
Screenshot 1

MainActivity.java code alongside the emulator with the name "Vimal" entered before clicking Greet.



Screenshot 2

Emulator output after clicking Greet – the greeting "Hello, Vimal!" is displayed.



Task 6 - Modify Task 5: Button and Text Colours

The button background was set to dark green with black text using **android:backgroundTint** and **android:textColor**. The greeting TextView's text colour was set to red using **android:textColor**.

```
res/layout/activity_main.xml

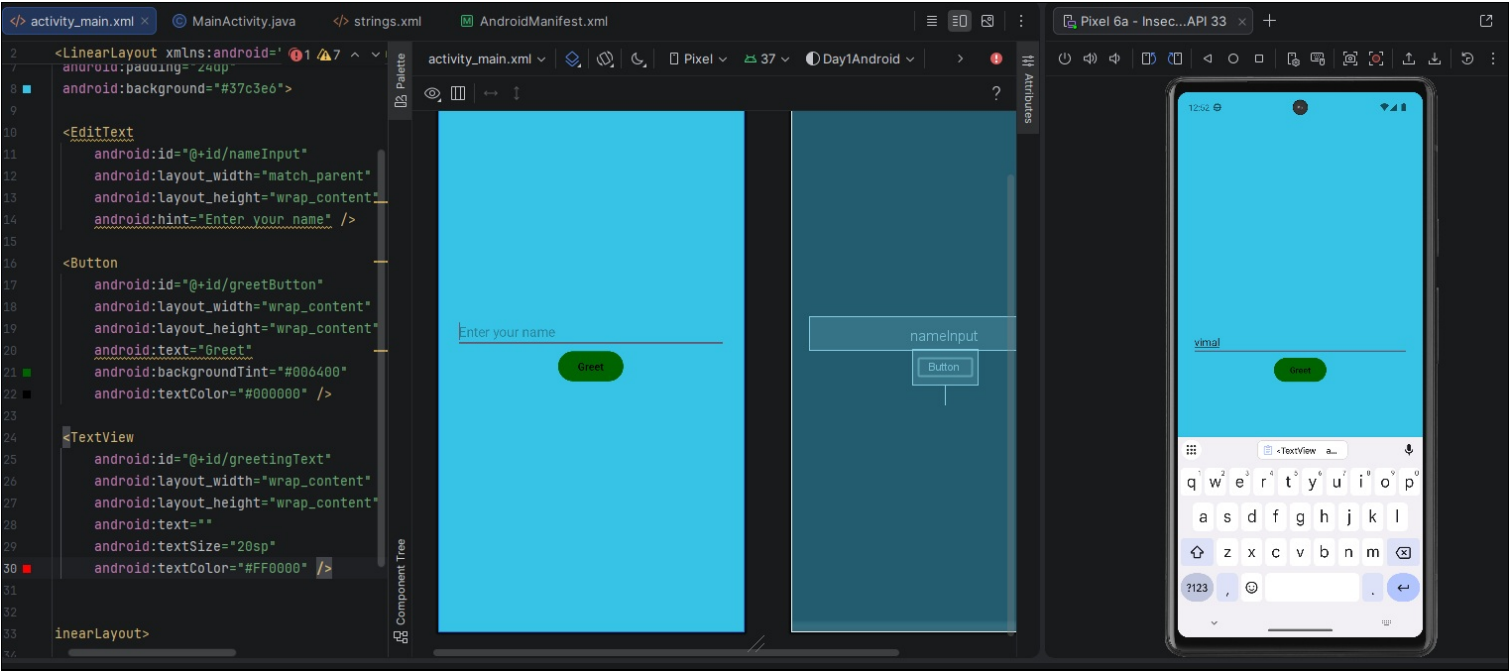
<EditText
    android:id="@+id/nameInput"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter your name" />

<Button
    android:id="@+id/greetButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Greet"
    android:backgroundTint="#006400"
    android:textColor="#000000" />

<TextView
    android:id="@+id/greetingText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text=""
    android:textSize="20sp"
    android:textColor="#FF0000" />
```

Screenshot

Design preview and emulator showing the dark green Greet button with black text, and the red greeting text after entering the name "vimal".



Task 7 - Random Dice Roller

The Random class generates a number between 1 and 6 each time the button is clicked, and the result is updated in a TextView inside **setOnClickListener()**.

res/layout/activity_main.xml

```
<TextView
    android:id="@+id/resultText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Roll the dice!"
    android:textSize="20sp" />

<Button
    android:id="@+id/rollButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Roll Dice" />
```

MainActivity.java

```
import java.util.Random;

public class MainActivity extends AppCompatActivity {

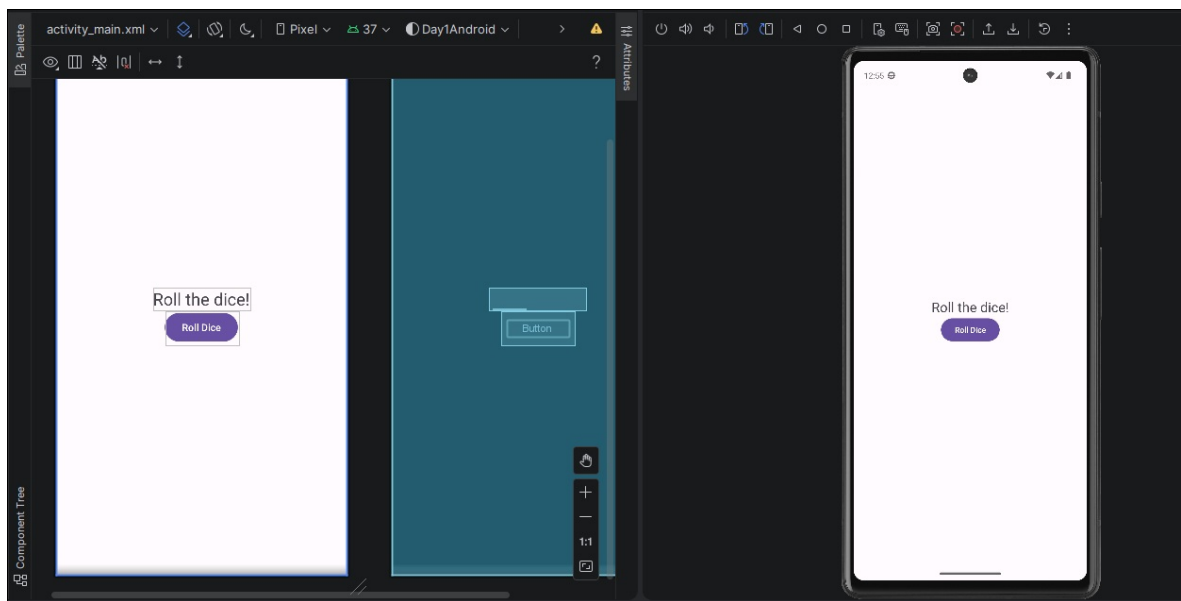
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        TextView resultText = findViewById(R.id.resultText);
        Button rollButton = findViewById(R.id.rollButton);
        Random random = new Random();

        rollButton.setOnClickListener(view -> {
            int roll = random.nextInt(6) + 1;
            resultText.setText("You rolled: " + roll);
        });
    }
}
```

Screenshot 1

Design view and emulator showing the initial "Roll the dice!" screen with the Roll Dice button.



Screenshot 2

Emulator output after clicking Roll Dice - a random result, "You rolled: 4", is displayed.

12:55



You rolled: 4

Roll Dice

Task 8 - Simple Calculator

Two EditText fields accept numeric input, which is safely parsed using **toDoubleOrNull()**-style conversion. Four buttons perform addition, subtraction, multiplication and division, updating the Result TextView based on the selected operation.

```
res/layout/activity_main.xml

<EditText android:id="@+id/num1" android:hint="Enter first number" />
<EditText android:id="@+id/num2" android:hint="Enter second number" />

<Button android:id="@+id/btnAdd" android:text="+" />
<Button android:id="@+id/btnSub" android:text="-" />
<Button android:id="@+id/btnMul" android:text="x" />
<Button android:id="@+id/btnDiv" android:text="÷" />

<TextView android:id="@+id/resultText" android:text="Result" />
```

```
MainActivity.java

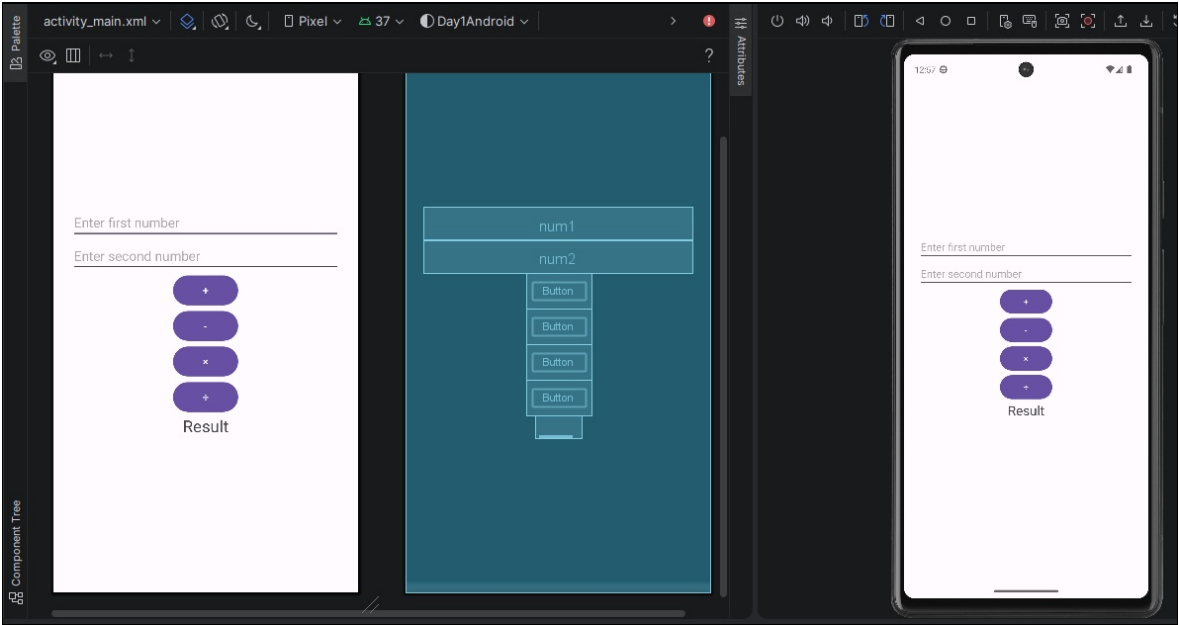
private void calculate(char operator) {
    double num1 = Double.parseDouble(editNum1.getText().toString());
    double num2 = Double.parseDouble(editNum2.getText().toString());
    double result = 0;

    switch (operator) {
        case '+': result = num1 + num2; break;
        case '-': result = num1 - num2; break;
        case '*': result = num1 * num2; break;
        case '/':
            result = (num2 != 0) ? num1 / num2 : 0;
            break;
    }
    resultText.setText("Result: " + result);
}

btnAdd.setOnClickListener(v -> calculate('+'));
btnSub.setOnClickListener(v -> calculate('-'));
btnMul.setOnClickListener(v -> calculate('*'));
btnDiv.setOnClickListener(v -> calculate('/'));
```

Screenshot 1

Design view and emulator showing the calculator layout with two input fields and four operation buttons.



Screenshot 2

Emulator output for inputs 20 and 50 with multiplication selected - Result: 1000.0.

12:58



20

50

+

-

×

÷

Result: 1000.0

1

2

3

-

4

5

6

=

7

8

9

✕

,

0

.

✓



Task 9 - Show / Hide TextView

A button toggles the visibility of a TextView using **setVisibility()**, switching between **View.VISIBLE** and **View.GONE** and updating the button label accordingly.

res/layout/activity_main.xml

<TextView
 android:id="@+id/hideText"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="Hello! This text can be hidden." />

<Button
 android:id="@+id/toggleButton"
 android:layout_width="wrap_content"
 android:layout_height="wrap_content"
 android:text="Hide Text" />

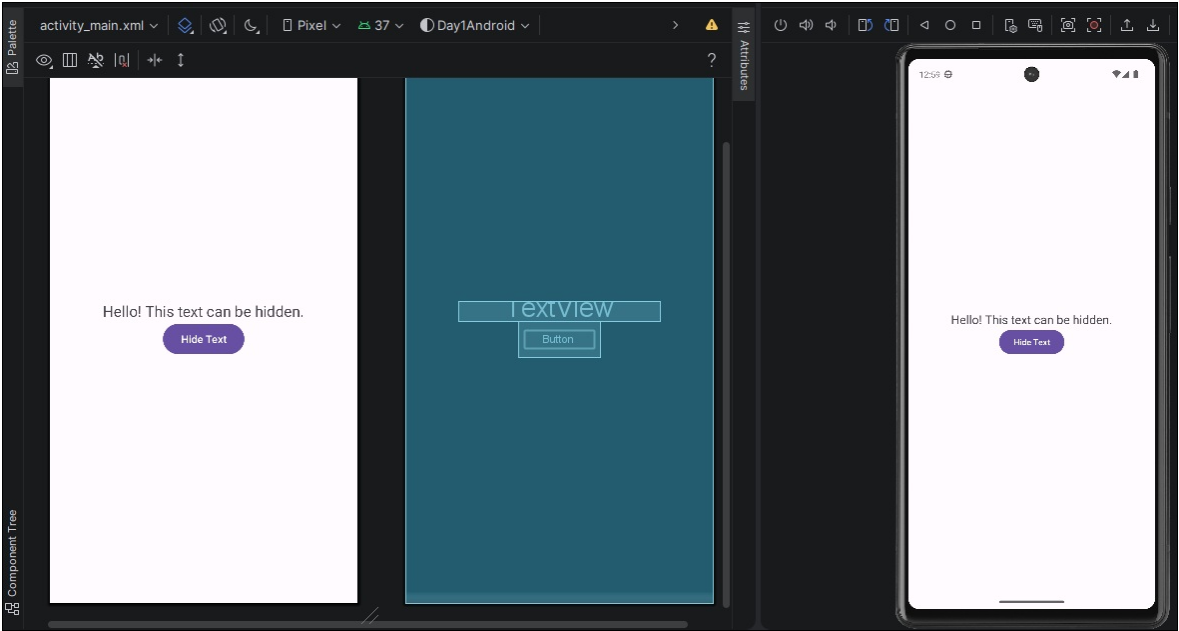
MainActivity.java

TextView hideText = findViewById(R.id.hideText);
Button toggleButton = findViewById(R.id.toggleButton);

toggleButton.setOnClickListener(view -> {
 if (hideText.getVisibility() == View.VISIBLE) {
 hideText.setVisibility(View.GONE);
 toggleButton.setText("Show Text");
 } else {
 hideText.setVisibility(View.VISIBLE);
 toggleButton.setText("Hide Text");
 }
});

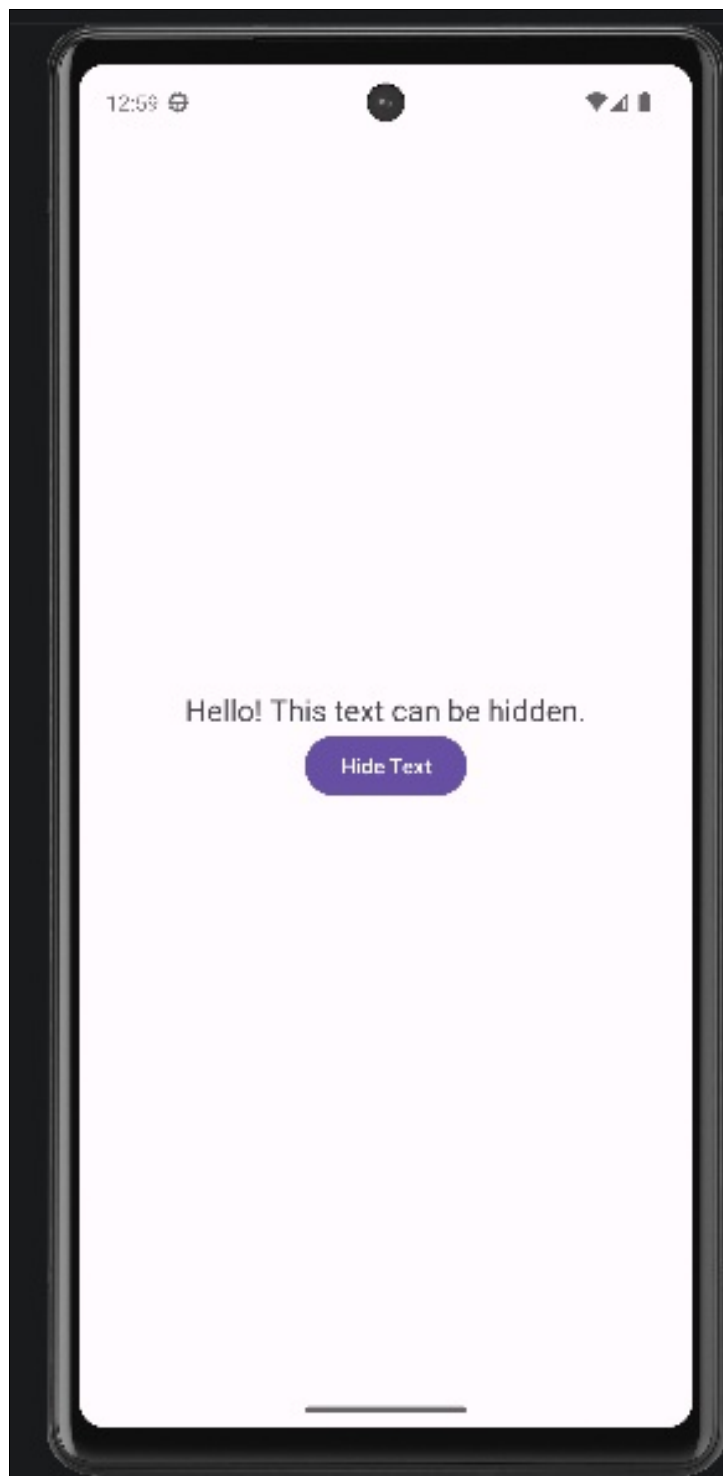
Screenshot 1

Design view and emulator showing the TextView visible with the "Hide Text" button.



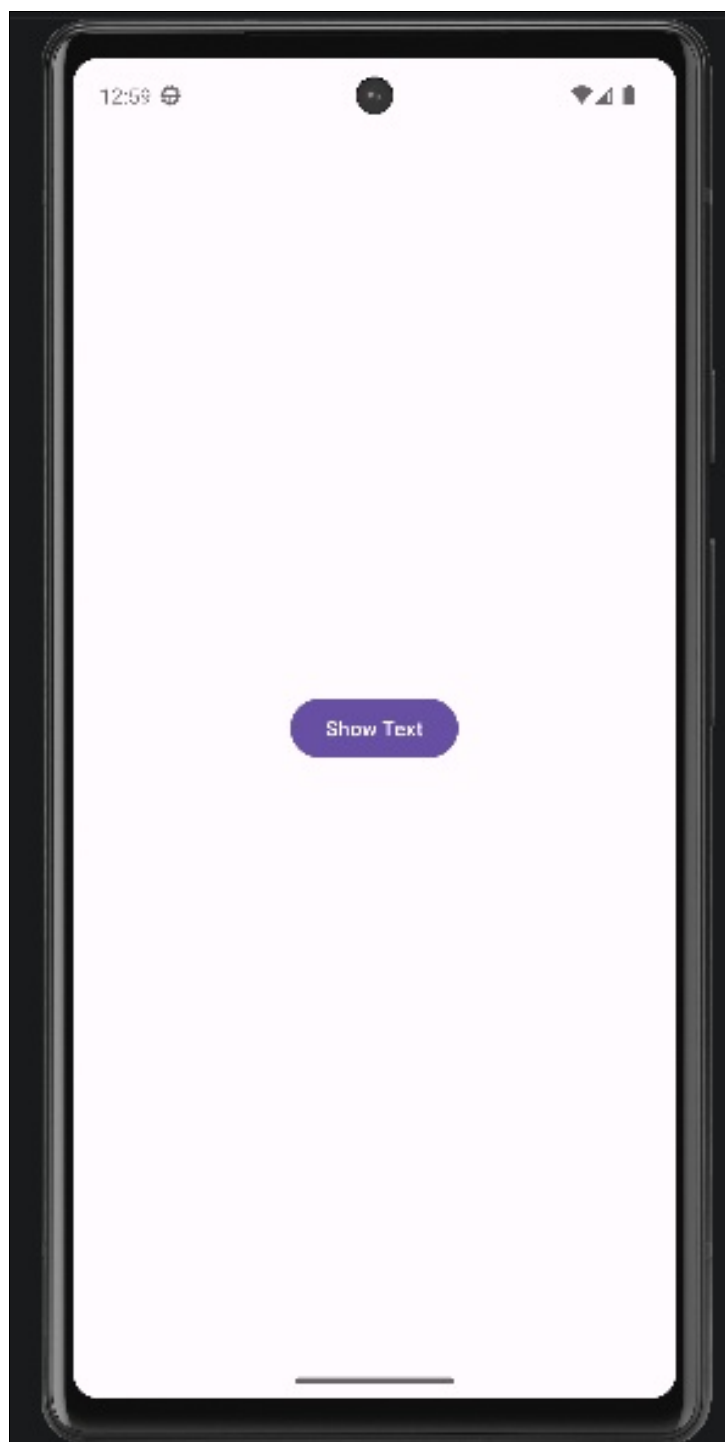
Screenshot 2

Emulator showing the initial state – text visible, button reads "Hide Text".



Screenshot 3

Emulator after clicking the button - the text is hidden and the button now reads "Show Text".



Task 10 - Select Favorite Programming Language

Three RadioButtons (Java, Python, C++) are grouped inside a RadioGroup. On clicking Submit, **getCheckedRadioButtonId()** retrieves the selected option and displays it in a TextView.

```
res/layout/activity_main.xml

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Choose your favorite language:" />

<RadioGroup
    android:id="@+id/languageGroup"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <RadioButton
        android:id="@+id/radioJava"
        android:text="Java" />

    <RadioButton
        android:id="@+id/radioPython"
        android:text="Python" />

    <RadioButton
        android:id="@+id/radioCpp"
        android:text="C++" />

</RadioGroup>

<Button
    android:id="@+id/submitButton"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Submit" />

<TextView
    android:id="@+id/selectedText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />
```

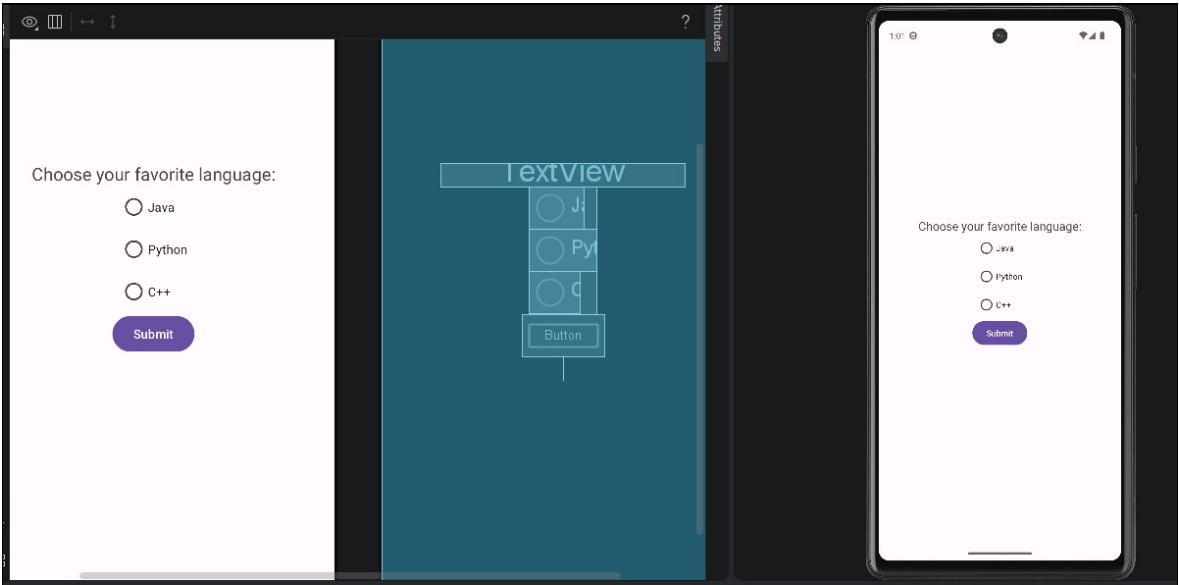
```
MainActivity.java

RadioGroup languageGroup = findViewById(R.id.languageGroup);
Button submitButton = findViewById(R.id.submitButton);
TextView selectedText = findViewById(R.id.selectedText);

submitButton.setOnClickListener(view -> {
    int selectedId = languageGroup.getCheckedRadioButtonId();
    RadioButton selectedButton = findViewById(selectedId);
    selectedText.setText("Selected: " + selectedButton.getText());
});
```

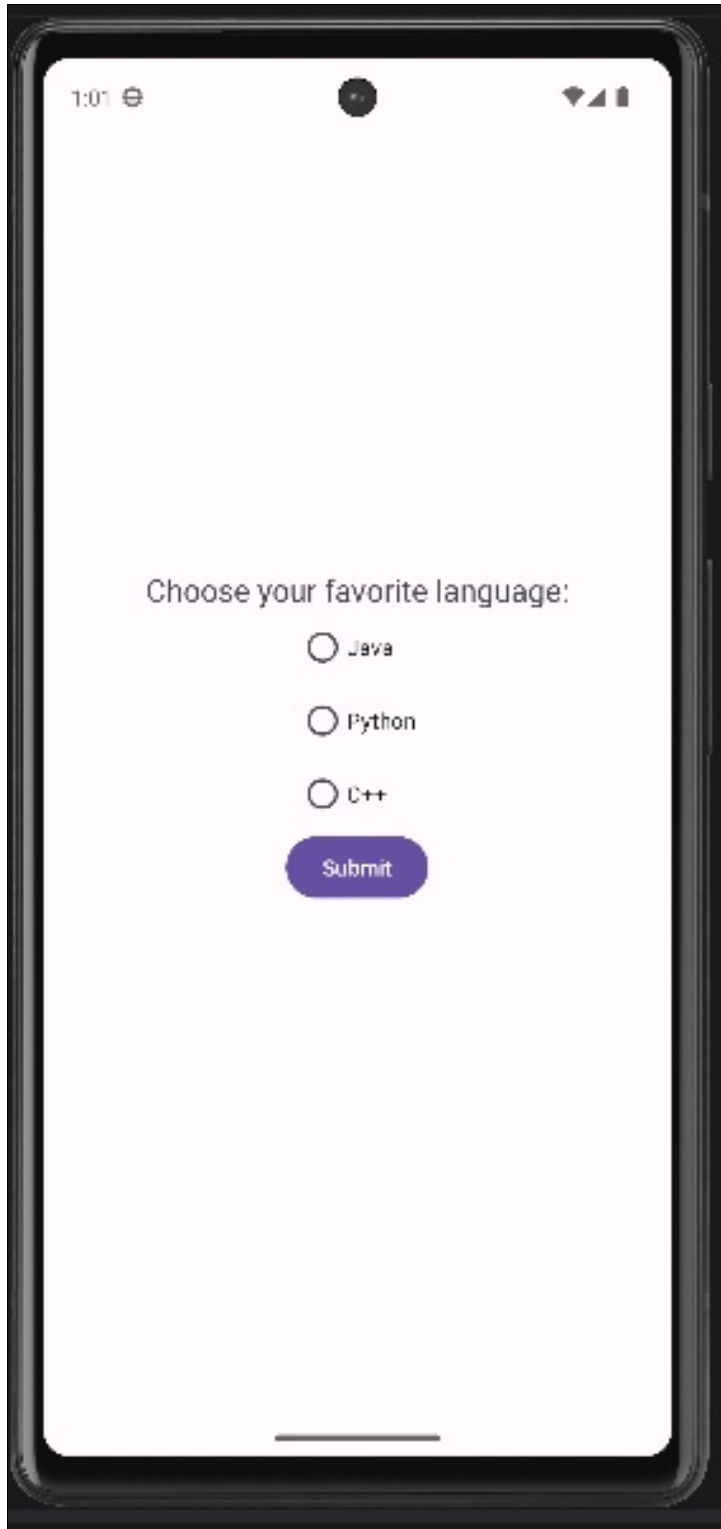
Screenshot 1

Design view and emulator showing the three RadioButtons with the Submit button.



Screenshot 2

Emulator showing the initial state before any language is selected.



Screenshot 3

Emulator after selecting "Python" and clicking Submit - "Selected: Python" is displayed.

1:01



Choose your favorite language:

☐ Java

☒ Python

☐ C++

Submit

Selected: Python